

Estrutura de Dados II

Revisão de Conceitos

Prof. Jobson Massollar
jobson.silva@uva.br

- Ponteiros
- Alocação Dinâmica de Memória
- Estruturas

- Todas as variáveis que declaramos em nossos programas ocupam, em algum momento, algum **endereço** na memória do computador.
- Variáveis são **nomes simbólicos** que facilitam a programação, já que não precisamos lidar diretamente com endereços da memória.
- Exemplo:

```
// a e b são variaveis do  
// tipo int  
int a;  
int b;
```

a: endereço 100
b: endereço 700

		?		
	?			

- Quando manipulamos (lemos ou escrevemos) uma variável, estamos manipulando o endereço de memória associado aquela variável.

- Exemplo:

```
int a;  
int b;  
  
a = 10;  
b = a;  
a = a + b;
```

Armazena **10** no
endereço **100**.

a: endereço 100

b: endereço 700

		10		
	?			

- Quando manipulamos (lemos ou escrevemos) uma variável, estamos manipulando o endereço de memória associado aquela variável.

- Exemplo:

```
int a;  
int b;  
  
a = 10;  
b = a;  
a = a + b;
```

Armazena o conteúdo
do **endereço 100** no
endereço 700.

a: endereço 100

b: endereço 700

		10		
	10			

- Quando manipulamos (lemos ou escrevemos) uma variável, estamos manipulando o **endereço de memória** associado aquela variável.

- Exemplo:

```
int a;
```

```
int b;
```

```
a = 10;
```

```
b = a;
```

```
a = a + b;
```

Soma os conteúdos do **endereço 100** e **700** e armazena o resultado no **endereço 700**.

a: endereço 100

b: endereço 700

		20		
	10			

- Ponteiros são variáveis especiais que **não armazenam dados**.
- Ponteiros armazenam o **endereço** de outras variáveis, por isso dizemos que ponteiros **apontam** ou **referenciam** outras variáveis.
- Para criar uma variável ponteiro basta colocar o ***** antes do seu nome.
- Exemplos:

```
// p1 é um ponteiro para uma variável do tipo int  
int *p1;
```

```
// p2 é um ponteiro para uma variável do tipo char  
char *p2;
```

```
// p3 é um ponteiro para uma variável do tipo float  
float *p3;
```

- Note que uma variável ponteiro ocupa um **endereço** de memória como qualquer outra, afinal uma variável ponteiro também é uma **variável**!

```
// a e b são variaveis  
// do tipo int  
int a, b;  
  
// p e um ponteiro para  
// uma variavel do tipo int  
int *p;
```

a: endereço 100

b: endereço 700

p: endereço 900

		?		
	?		?	

➤ Como usamos a variável **p**?

```
// a e b são variaveis do tipo int
int a, b;
```

```
// p e um ponteiro para
// uma variavel do tipo int
int *p;
```

```
a = 10;
```

a: endereço 100
b: endereço 700
p: endereço 900

		10		
	?		?	

- O operador `&` retorna o endereço de uma variável.

```
// a e b são variaveis do tipo int  
int a, b;
```

```
// p é um ponteiro para  
// uma variavel do tipo int  
int *p;
```

```
a = 10;
```

```
p = &a;
```

O operador `&` retorna o endereço da variável `a` e não seu conteúdo.

a: endereço 100
b: endereço 700
p: endereço 900

		10		
	?		?	

- O operador & retorna o endereço de uma variável.

```
// a e b são variáveis do tipo int  
int a, b;
```

```
// p é um ponteiro para  
// uma variável do tipo int  
int *p;
```

```
a = 10;
```

```
p = &a;
```

Variáveis ponteiro recebem o **endereço** de outras variáveis.
Assim, **p** vai receber o **endereço de a** e não o conteúdo de a.

a: endereço 100
b: endereço 700
p: endereço 900

		10		
	?		100	

- Uma variável ponteiro **referencia** ou **aponta** para outra variável.

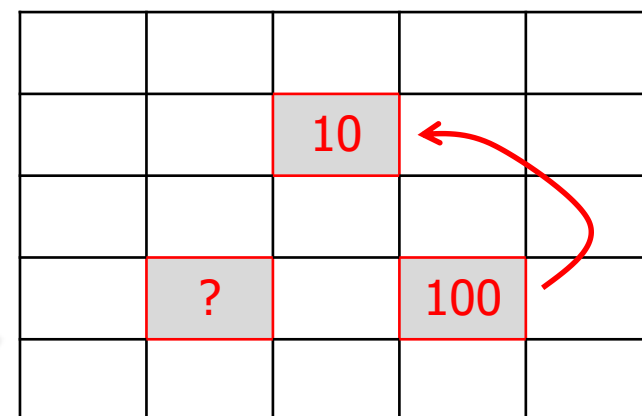
```
// a e b são variaveis do tipo int  
int a, b;
```

```
// p é um ponteiro para  
// uma variavel do tipo int  
int *p;
```

```
a = 10;
```

```
p = &a;
```

a: endereço 100
b: endereço 700
p: endereço 900



Agora podemos dizer que
p aponta para a
ou
p referencia a.

- O operador * recupera o valor referenciado por um ponteiro

```
// a e b são variaveis do tipo int
```

```
int a, b;
```

```
// p é um ponteiro para
```

```
// uma variavel do tipo int
```

```
int *p;
```

```
a = 10;
```

```
p = &a;
```

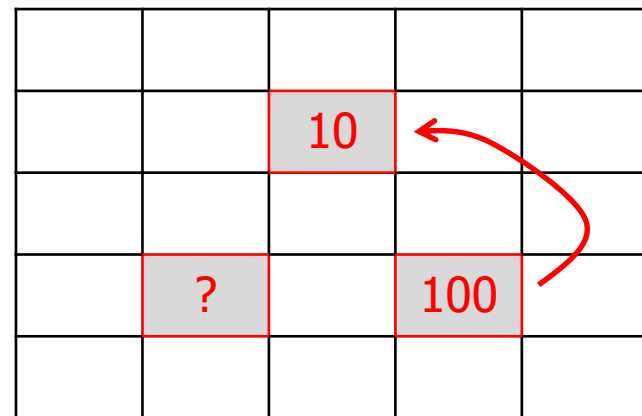
```
b = *p;
```

O operador * retorna o valor referenciado ou apontado pelo ponteiro **p**.

a: endereço 100

b: endereço 700

p: endereço 900



- O operador * recupera o **valor referenciado** por um ponteiro

```
// a e b são variaveis do tipo int
```

```
int a, b;
```

```
// p é um ponteiro para
```

```
// uma variavel do tipo int
```

```
int *p;
```

```
a = 10;
```

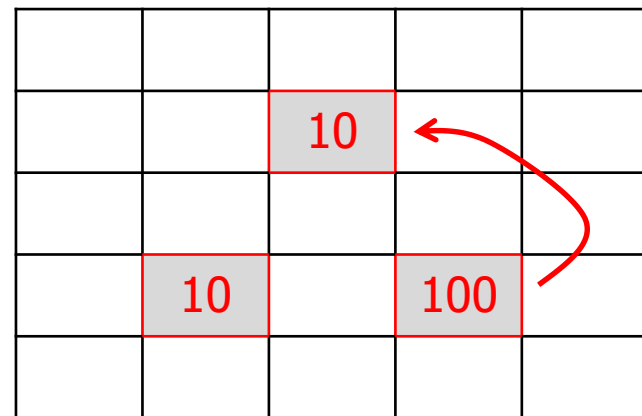
```
p = &a;
```

```
b = *p;
```

a: endereço 100

b: endereço 700

p: endereço 900



Assim, a variável **b** vai receber o valor referenciado por **p**, ou seja, **b** vai receber **10**.

- O operador * recupera o **valor referenciado** por um ponteiro

```
// a e b são variaveis do tipo int
```

```
int a, b;
```

```
// p é um ponteiro para
```

```
// uma variavel do tipo int
```

```
int *p;
```

```
a = 10;
```

```
p = &a;
```

```
b = *p;
```

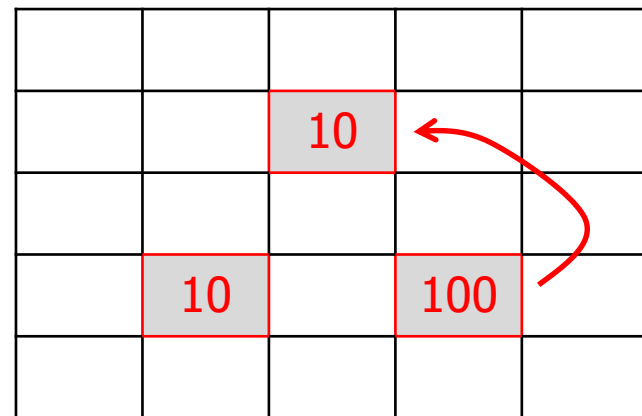
Soma o valor referenciado pelo ponteiro **p** com a variável **b** e armazena o resultado no endereço de memória referenciado pelo ponteiro **p**.

```
*p = *p + b;
```

a: endereço 100

b: endereço 700

p: endereço 900



- O operador * recupera o **valor referenciado** por um ponteiro

```
// a e b são variaveis do tipo int
```

```
int a, b;
```

```
// p é um ponteiro para
```

```
// uma variavel do tipo int
```

```
int *p;
```

```
a = 10;
```

```
p = &a;
```

```
b = *p;
```

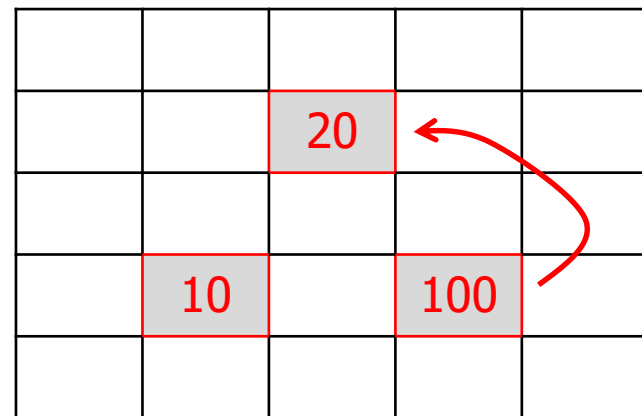
O efeito final é o mesmo que
a = a + b

```
*p = *p + b;
```

a: endereço 100

b: endereço 700

p: endereço 900



- Quando atribuímos **NULL** a um ponteiro estamos dizendo que ele não referencia **nada**.

```
// a e b são variaveis do tipo int  
int a, b;
```

```
// p é um ponteiro para  
// uma variavel do tipo int  
int *p;
```

```
a = 10;  
p = &a;  
b = *p;  
*p = *p + b;  
P = NULL;
```

a: endereço 100
b: endereço 700
p: endereço 900

		20		
	10		NULL	

O ponteiro **p** não
referencia nenhuma
variável.

- Só faz sentido utilizar dois operadores relacionais com ponteiros:

Operador	Significado
==	Verifica se dois ponteiros referenciam o mesmo endereço de memória ou NULL
!=	Verifica se dois ponteiros referenciam diferentes endereços de memória.

➤ Exemplos:

```
int a = 10, b = 20;  
int *p1 = &a, *p2 = &b;
```

```
if (p1 == p2)  
    puts("P1 e P2 referenciam o mesmo endereco");
```

```
if (p1 != p2)  
    puts("P1 e P2 referenciam enderecos distintos");
```

```
if (p1 == NULL)  
    puts("P1 não referencia nenhum endereco de memoria");
```

```
if (p2 != NULL)  
    puts("P2 referencia algum endereco de memoria");
```

- Vimos anteriormente que quando passamos tipos primitivos para uma função, seus valores **não podem ser alterados**, pois a passagem de parâmetros é por **valor**.
- Entretanto, se passarmos um **ponteiro para variável**, a função será capaz de **alterar o seu valor**.
- Nesse caso, temos o que chamamos de passagem de parâmetros por **referência**.

➤ Exemplo:

```
void troca(int *x, int *y)
{
    int temp = *x;
    *x = *y;
    *y = temp;
}
```

Note que os
parâmetros **x** e **y** não
são inteiros, eles são
ponteiros para
inteiros.

```
int main(void)
{
    int a = 10, b = 20;

    troca(&a, &b);

    printf("%d  %d\n", a, b);
}
```

➤ Exemplo:

```
void troca(int *x, int *y)
{
    int temp = *x;
    *x = *y;
    *y = temp;
}
```

```
int main(void)
{
    int a = 10, b = 20;

    troca(&a, &b);

    printf("%d  %d\n", a, b);
}
```

a

10

b

20

Aloca e
inicializa **a** e **b**

➤ Exemplo:

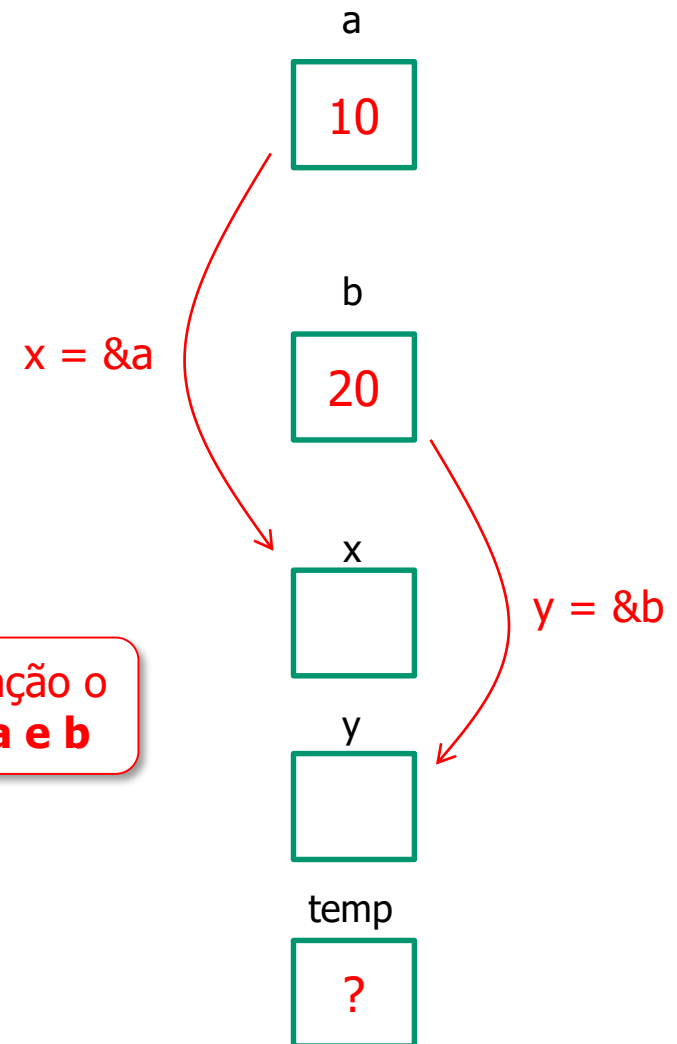
```
void troca(int *x, int *y)
{
    int temp = *x;
    *x = *y;
    *y = temp;
}
```

```
int main(void)
{
    int a = 10, b = 20;

    troca(&a, &b);

    printf("%d  %d\n", a, b);
}
```

Passa para a função o endereço de a e b



➤ Exemplo:

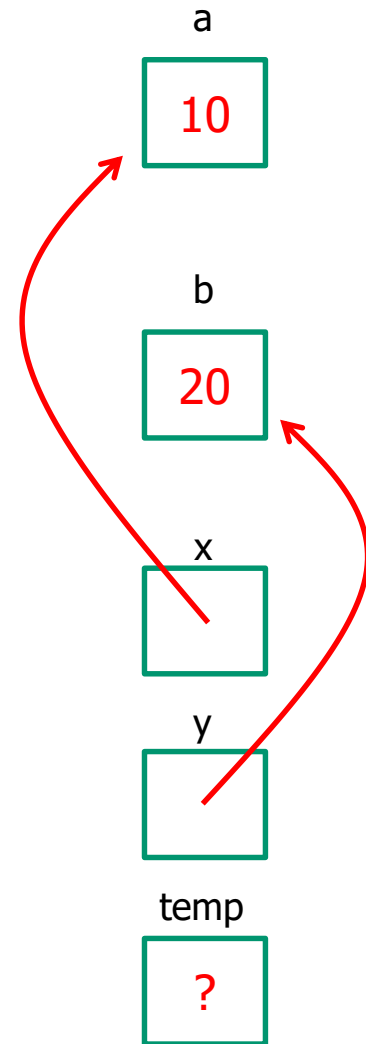
```
void troca(int *x, int *y)
{
    int temp = *x;
    *x = *y;
    *y = temp;
}
```

Assim:
x aponta para a
y aponta para b

```
int main(void)
{
    int a = 10, b = 20;

    troca(&a, &b);

    printf("%d  %d\n", a, b);
}
```



➤ Exemplo:

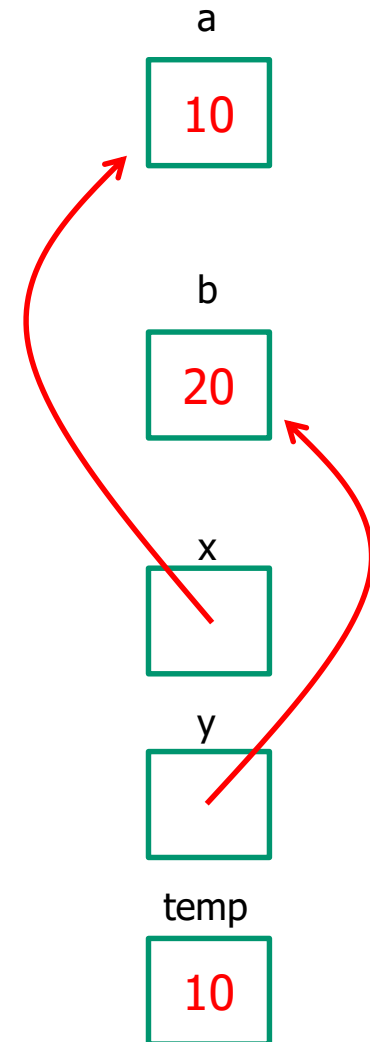
```
void troca(int *x, int *y)
{
    int temp = *x;
    *x = *y;
    *y = temp;
}
```

```
int main(void)
{
    int a = 10, b = 20;

    troca(&a, &b);

    printf("%d  %d\n", a, b);
}
```

temp recebe o valor referenciado pelo ponteiro **x**.



➤ Exemplo:

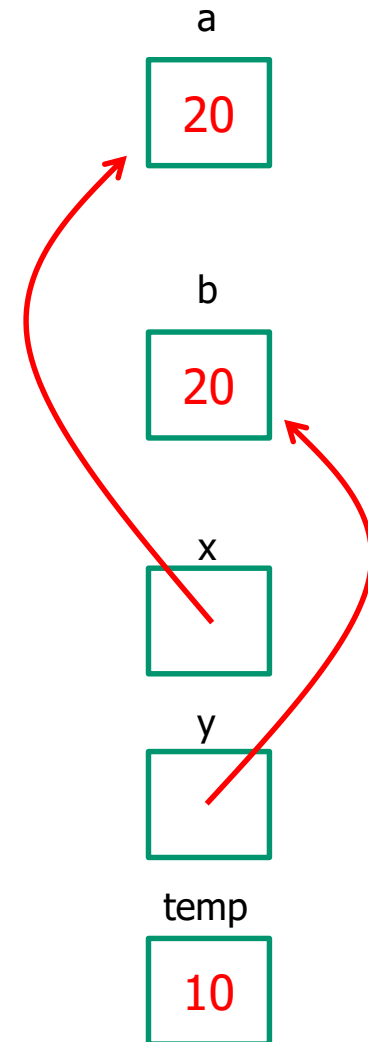
```
void troca(int *x, int *y)
{
    int temp = *x;
    *x = *y;
    *y = temp;
}
```

```
int main(void)
{
    int a = 10, b = 20;

    troca(&a, &b);

    printf("%d  %d\n", a, b);
}
```

O endereço de memória referenciado por **x** recebe o valor referenciado por **y**.



➤ Exemplo:

```
void troca(int *x, int *y)
```

```
{
```

```
    int temp = *x;
```

```
    *x = *y;
```

```
    *y = temp;
```

```
}
```

```
int main(void)
```

```
{
```

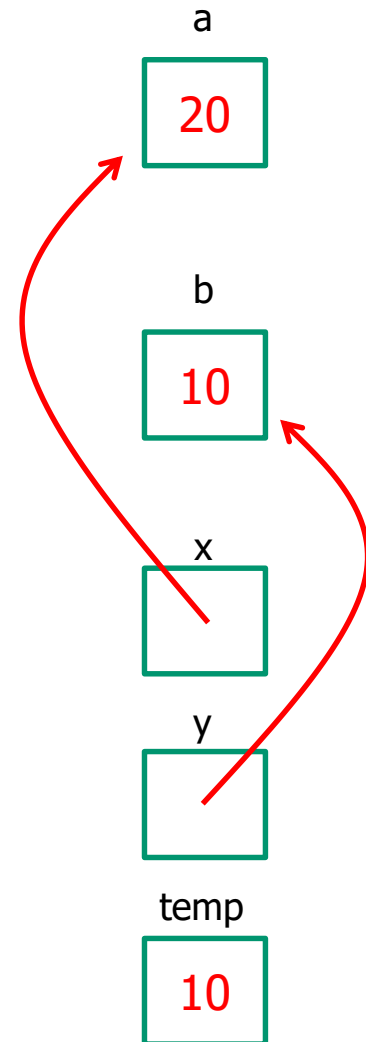
```
    int a = 10, b = 20;
```

```
    troca(&a, &b);
```

```
    printf("%d  %d\n", a, b);
```

```
}
```

O endereço de memória referenciado por **y** recebe o valor de **temp**.



➤ Exemplo:

```
void troca(int *x, int *y)
{
    int temp = *x;
    *x = *y;
    *y = temp;
}
```

```
int main(void)
{
    int a = 10, b = 20;

    troca(&a, &b);

    printf("%d  %d\n", a, b);
}
```

a

20

b

10

Os parâmetros e a variável da função **troca** são desalocados e o valor de **a** foi trocado com **b**.

- Na linguagem C/C++:
 1. Variáveis declaradas **fora das funções** tem **escopo global**, ou seja, podem ser manipuladas por todas as funções. Por isso, essas variáveis são chamadas **globais**.
 2. Variáveis declaradas **dentro das funções ou dentro de blocos** de comando tem escopo **local**, ou seja, só podem ser manipuladas dentro da função ou bloco que as declara. Por isso são chamadas de variáveis **locais**.
- **Observação:** os **parâmetros** das funções são tratados como variáveis **locais**, ou seja, só podem ser manipulados **dentro da função**.

- Do ponto de vista da alocação de memória:
 1. As variáveis **globais** são alocadas na memória quando o Sistema Operacional aloca espaço para execução do programa, ou seja, imediatamente antes do programa iniciar a sua execução.
 2. As variáveis **globais** são desalocadas da memória somente quando o programa **termina**.
 3. As variáveis **globais** são alocadas na memória em uma área chamada **segmento de dados** (*data segment*).
 4. Se declararmos uma variável **global** muito grande (por exemplo, um vetor com vários gigabytes de tamanho) e não houver espaço suficiente no **segmento de dados** para essa variável, o Sistema Operacional irá avisar que **não há espaço suficiente** para executar o programa.

- Do ponto de vista da alocação de memória:
 1. As variáveis **locais** são alocadas na memória somente quando a função é executada.
 2. As variáveis **locais** são desalocadas da memória quando a função **termina**.
 3. As variáveis **locais** são alocadas na memória em uma área chamada **segmento de pilha** (*stack segment*).
 4. Se declararmos uma variável **local** muito grande (por exemplo, um vetor com vários gigabytes de tamanho) e não houver espaço suficiente na **pilha** para essa variável, o Sistema Operacional irá retornar um erro de **estouro de pilha** (*stack overflow*).

- A questão é que muitas vezes precisamos alocar grandes espaços de memória para executar somente algumas tarefas dentro de um programa.
- Nesse caso, seria interessante poder alocar o espaço na memória no momento em que ele for efetivamente necessário e desalocá-lo quando ele não for mais necessário.
- Isso é possível através da **alocação dinâmica de memória**.
- Assim, alocação dinâmica de memória é a possibilidade de, durante a execução de um programa, solicitar ao Sistema Operacional uma área extra de memória que não foi declarada em nenhuma variável.
- Entretanto, como o nosso programa vai trabalhar com uma área de memória se não temos uma variável declarada para referenciar essa memória ? Resposta: **usando ponteiros**.

- Para alocar memória em programas C usamos a função:

```
void *malloc(long tam)
```

- Essa função aloca um espaço de **tam** bytes na memória e retorna o **endereço da área alocada**. Usando uma variável ponteiro, podemos guardar esse endereço para usá-la posteriormente. Exemplo:

```
int *p;
```

```
p = malloc(10240);
```

Aloca 10Kb de memória dinamicamente e retorna o endereço da memória que é guardado na variável **p**.

Dizemos, então, que **p** aponta para a área de memória alocada.

Caso não exista espaço suficiente na memória, a função malloc() retorna **ZERO** ou **NULL**.

- Para desalocar memória em programas C usamos a função:

```
void free(void *p)
```

- Essa função desaloca a área de memória apontada pela variável `p`.
Exemplo:

```
int *p;
```

```
p = malloc(10240);
```

```
free(p);
```

Os 10Kb alocados anteriormente são retornados para o Sistema Operacional, que poderá alocar essa área de memória para outros programas.

- Programas em C++ podem usar as funções `malloc()` e `free()` ou podem usar dois novos comandos para alocar e desalocar a memória: `new` e `delete`.
- Os comandos `new` e `delete` tem sintaxe diferente das funções `malloc()` e `free()`, mas tem exatamente o mesmo propósito.
- Esse comando foram criados para trabalhar com objetos no C++.

- Por exemplo: imagine que você deve criar um programa para ler a matricula, o salário e o número de dependentes de um funcionário.
- Uma possível solução seria:

```
int main() {  
    int matricula, num_deps;  
    float salario;  
  
    printf("Matricula: ");  
    scanf("%d", &matricula);  
    printf("Salario: ");  
    scanf("%f", &salario);  
    printf("No. de dependentes: ");  
    scanf("%d", &num_deps);  
}
```

- Entretanto, note que essas três informações pertencem a **um funcionário**, ou seja, **todo funcionário possui essas três informações**.
- Então, porque não agrupá-las para representar os dados de um funcionário?
- Para isso existem as **estruturas** ou **structs** em C/C++.
- Cada "parte" de uma estrutura representa um **dado**, também chamado **campo da estrutura**.

➤ Sintaxe:

```
struct NOME_DA_ESTRUTURA {  
    <tipo> nome_do_campo_1;  
    <tipo> nome_do_campo_2;  
    <tipo> nome_do_campo_3;  
    ...  
};
```

➤ Note que cada campo da **struct** :

- ✓ É declarado usando a **mesma sintaxe** da **declaração de uma variável**.
- ✓ Pode ter um tipo distinto.

- Definindo uma **struct** para o funcionário:

```
struct FUNCIONARIO {  
    int matricula;  
    float salario;  
    int num_deps;  
};
```

- A partir da definição da **struct** podemos:
 - ✓ Criar variáveis do tipo da **struct**;
 - ✓ Acessar **individualmente** os campos da **struct**.

➤ Exemplo 1:

```
struct FUNCIONARIO {  
    int matricula;  
    float salario;  
    int num_deps;  
};
```

```
int main() {  
    struct FUNCIONARIO f;  
  
    f.matricula = 243564;  
    f.salario = 5450.0;  
    f.num_deps = 2;  
}
```

Para criar uma variável basta usar a sintaxe:

struct NOME_DA ESTRUTURA var;

	f
matricula	?
salario	?
num_deps	?

➤ Exemplo 1:

```
struct FUNCIONARIO {  
    int matricula;  
    float salario;  
    int num_deps;  
};  
  
int main() {  
    struct FUNCIONARIO f;  
  
    f.matricula = 243564;  
    f.salario = 5450.0;  
    f.num_deps = 2;  
}
```

Para acessar um campo da estrutura a partir de uma variável basta usar a sintaxe:

var.NOME_DO_CAMPO

f	
matricula	243564
salario	5450.0
num_deps	2

➤ Exemplo 2:

```
struct FUNCIONARIO {  
    int matricula;  
    float salario;  
    int num_deps;  
};  
  
int main() {  
    struct FUNCIONARIO f;  
  
    printf("Matricula: ");  
    scanf("%d", &f.matricula);  
    printf("Salario: ");  
    scanf("%f", &f.salario);  
    printf("No. de dependentes: ");  
    scanf("%d", &f.num_deps);  
}
```

Para ler os campos de uma **struct** basta usar o operador **&** normalmente.

- Note que é possível ter:
 1. Um vetor como um **campo** de uma estrutura

```
struct ALUNO {  
    int matricula;  
  
    // Vetor de CHAR para armazenar o nome do aluno  
    char nome[20];  
  
    // Vetor de FLOAT para armazenar as 3 notas do aluno  
    float notas[3];  
};
```

➤ Note que é possível ter:

1. Um vetor como um **campo** de uma estrutura

```
struct ALUNO {  
    int matricula;  
    char nome[20];  
    float notas[3];  
};  
  
int main() {  
    struct ALUNO a;  
  
    printf("Matricula: ");  
    scanf("%d", &a.matricula);  
    printf("Nome: ");  
    gets(a.nome);  
    for (int i = 0; i < 3; i++) {  
        printf("Nota: ");  
        scanf("%f", &a.notas[i]);  
    }  
}
```

	a		
matricula	1342876		
nome	"João da Silva"		
notas	5.5	8.2	7.4

- Note que é possível ter:
2. Um **vetor** de estrutura

```
struct ALUNO {
    int matricula;
    char nome[20];
    float notas[3];
};

int main() {
    struct ALUNO alunos[5];

    for (int i = 0; i < 5; i++) {
        printf("Matricula: ");
        scanf("%d", &alunos[i].matricula);
        printf("Nome: ");
        gets(alunos[i].nome);
        for (int j = 0; j < 3; j++) {
            printf("Nota: ");
            scanf("%f", &alunos[i].notas[j]);
        }
    }
}
```

alunos

matricula	1342876		
nome	"João da Silva"		
notas	5.5	8.2	7.4
matricula	543275		
nome	"Ana Paula"		
notas	6.2	9.7	7.6
matricula	645342		
nome	"Pedro Vieira"		
notas	3.1	4.5	2.8
matricula	234164		
nome	"Beatriz Amaral"		
notas	7.8	4.7	9.6
matricula	873546		
nome	"Carlos Peixoto"		
notas	2.6	5.8	7.8

- Note que é possível ter:
3. Uma estrutura como **parte** de outra estrutura.

```
struct PONTO {  
    float x;  
    float y;  
};  
  
struct RETANGULO {  
    struct PONTO p1;  
    struct PONTO p2;  
};  
  
int main() {  
    struct RETANGULO r;  
  
    r.p1.x = 3.5;  
    r.p1.y = 2.0;  
    r.p2.x = 8.7;  
    r.p2.y = 12.5;  
}
```

		r
p1	x	3.5
	y	2.0
p2	x	8.7
	y	12.5

- Note que é possível ter:
 3. Uma estrutura como **parte** de outra estrutura.

```
struct PONTO {  
    float x;  
    float y;  
};  
  
struct RETANGULO {  
    struct PONTO p1;  
    struct PONTO p2;  
};  
  
int main() {  
    struct RETANGULO r;  
  
    scanf("%f", &r.p1.x);  
    scanf("%f", &r.p1.y);  
    scanf("%f", &r.p2.x);  
    scanf("%f", &r.p2.y);  
}
```

➤ Outras características:

1. Quando atribuímos uma estrutura à outra, o C/C++ copia **automaticamente** todos os campos de uma para outra.

```
int main() {  
    struct FUNCIONARIO f1, f2;  
  
    printf("Matricula: ");  
    scanf("%d", &f1.matricula);  
    printf("Salario: ");  
    scanf("%f", &f1.salario);  
    printf("No. de dependentes: ");  
    scanf("%d", &f1.num_deps);  
  
    f2 = f1;  
}
```

f1	
matricula	243564
salario	5450.0
num_deps	2

f2	
matricula	?
salario	?
num_deps	?

➤ Outras características:

1. Quando atribuímos uma estrutura à outra, o C/C++ copia **automaticamente** todos os campos de uma para outra.

```
int main() {  
    struct FUNCIONARIO f1, f2;  
  
    printf("Matricula: ");  
    scanf("%d", &f1.matricula);  
    printf("Salario: ");  
    scanf("%f", &f1.salario);  
    printf("No. de dependentes: ");  
    scanf("%d", &f1.num_deps);
```

```
    f2 = f1;  
}
```

A variável f2 vai ter os mesmos valores da variável f1.

f1

matricula	243564
salario	5450.0
num_deps	2

f2

matricula	243564
salario	5450.0
num_deps	2

➤ Outras características:

2. Estruturas são sempre passadas para funções **por valor**.

```
void imprime(struct FUNCIONARIO func) {  
    printf("Matricula: %s\n", func.matricula);  
    printf("Salario: %s\n", func.salario);  
    printf("No. dependentes: %s\n", func.num_deps);  
}
```

```
int main() {  
    struct FUNCIONARIO f;  
  
    printf("Matricula: ");  
    scanf("%d", &f.matricula);  
    printf("Salario: ");  
    scanf("%f", &f.salario);  
    printf("No. de dependentes: ");  
    scanf("%d", &f.num_deps);  
  
    imprime(f);  
}
```

f	
matricula	243564
salario	5450.0
num_deps	2

➤ Outras características:

2. Estruturas são sempre passadas para funções **por valor**.

```
void imprime(struct FUNCIONARIO func) {  
    printf("Matricula: %s\n", func.matricula);  
    printf("Salario: %s\n", func.salario);  
    printf("No. dependentes: %s\n", func.num_deps);  
}
```

```
int main() {  
    struct FUNCIONARIO f;  
  
    printf("Matricula: ");  
    scanf("%d", &f.matricula);  
    printf("Salario: ");  
    scanf("%f", &f.salario);  
    printf("No. de dependentes: ");  
    scanf("%d", &f.num_deps);  
  
    imprime(f);  
}
```

f	
matricula	243564
salario	5450.0
num_deps	2

➤ Outras características:

2. Estruturas são sempre passadas para funções **por valor**.

```
void imprime(struct FUNCIONARIO func) {  
    printf("Matricula: %s\n", func.matricula);  
    printf("Salario: %s\n", func.salario);  
    printf("No. dependentes: %s\n", func.num_deps);  
}
```

```
int main() {  
    struct FUNCIONARIO f;  
  
    printf("Matricula: ");  
    scanf("%d", &f.matricula);  
    printf("Salario: ");  
    scanf("%f", &f.salario);  
    printf("No. de dependentes: ");  
    scanf("%d", &f.num_deps);  
  
    imprime(f);  
}
```

func

matricula	243564
salario	5450.0
num_deps	2

func = f

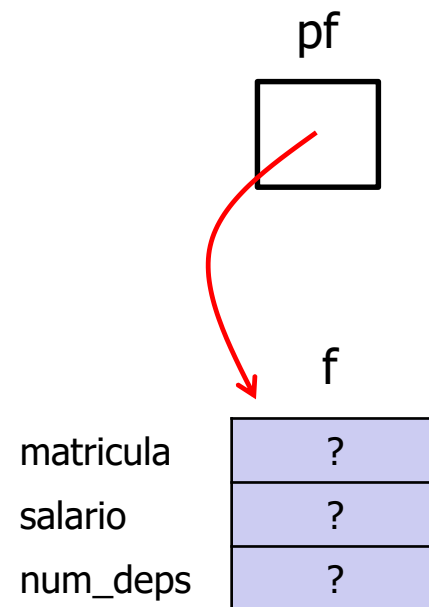
f

matricula	243564
salario	5450.0
num_deps	2

➤ Outras características:

3. É possível definir **ponteiros** para estruturas.

```
int main() {  
    struct FUNCIONARIO f;  
    struct FUNCIONARIO *pf;  
  
    pf = &f;  
  
    pf->matricula = 243564;  
    pf->salario = 5450.0;  
    pf->num_deps = 2;  
}
```



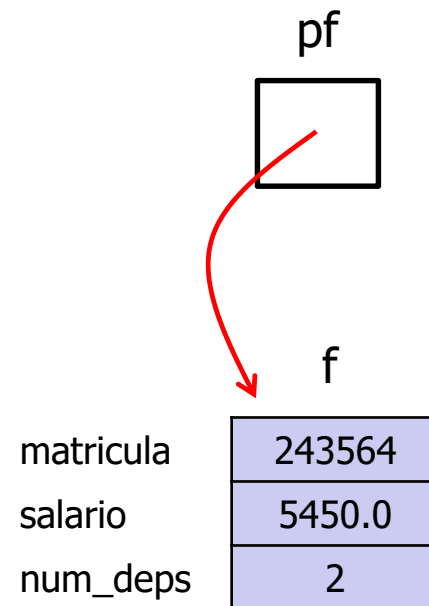
➤ Outras características:

3. É possível definir **ponteiros** para estruturas.

```
int main() {  
    struct FUNCIONARIO f;  
    struct FUNCIONARIO *pf;  
  
    pf = &f;  
  
    pf->matricula = 243564;  
    pf->salario = 5450.0;  
    pf->num_deps = 2;  
}
```

Para acessar um campo da estrutura a partir de um ponteiro basta usar a sintaxe:

ponteiro->NOME_DO_CAMPO



➤ Outras características:

4. Para passar uma estrutura **por referência** devemos usar ponteiros.

```
void ler(struct FUNCIONARIO *pf) {  
    printf("Matricula: ");  
    scanf("%d", &pf->matricula);  
    printf("Salario: ");  
    scanf("%f", &pf->salario);  
    printf("No. de dependentes: ");  
    scanf("%d", &pf->num_deps);  
}
```

```
int main() {  
    struct FUNCIONARIO f;  
  
    ler(&f);  
    printf("Matricula: %d\n",    f.matricula);  
    printf("Salario   : %.2f\n", f.salario);  
    printf("No. Deps  : %d\n",    f.num_deps);  
}
```

	f
matricula	?
salario	?
num_deps	?

➤ Outras características:

4. Para passar uma estrutura **por referência** devemos usar ponteiros.

```
void ler(struct FUNCIONARIO *pf) {  
    printf("Matricula: ");  
    scanf("%d", &pf->matricula);  
    printf("Salario: ");  
    scanf("%f", &pf->salario);  
    printf("No. de dependentes: ");  
    scanf("%d", &pf->num_deps);  
}
```

```
int main() {  
    struct FUNCIONARIO f;  
  
    ler(&f);  
  
    printf("Matricula: %d\n", f.matricula);  
    printf("Salario : %.2f\n", f.salario);  
    printf("No. Deps : %d\n", f.num_deps);  
}
```

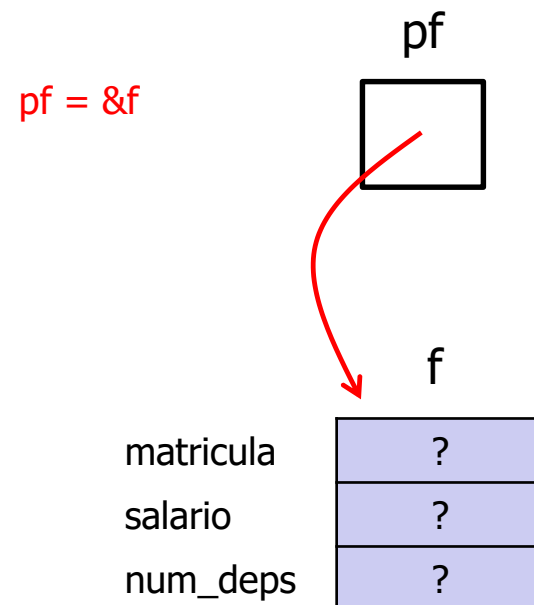
	f
matricula	?
salario	?
num_deps	?

➤ Outras características:

4. Para passar uma estrutura **por referência** devemos usar ponteiros.

```
void ler(struct FUNCIONARIO *pf) {  
    printf("Matricula: ");  
    scanf("%d", &pf->matricula);  
    printf("Salario: ");  
    scanf("%f", &pf->salario);  
    printf("No. de dependentes: ");  
    scanf("%d", &pf->num_deps);  
}
```

```
int main() {  
    struct FUNCIONARIO f;  
  
    ler(&f);  
  
    printf("Matricula: %d\n", f.matricula);  
    printf("Salario : %.2f\n", f.salario);  
    printf("No. Deps : %d\n", f.num_deps);  
}
```

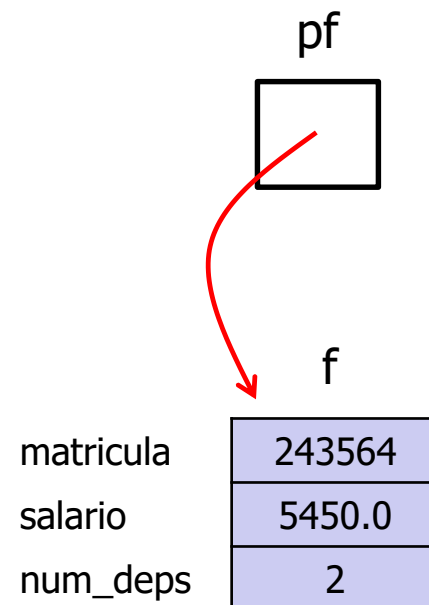


➤ Outras características:

4. Para passar uma estrutura **por referência** devemos usar ponteiros.

```
void ler(struct FUNCIONARIO *pf) {  
    printf("Matricula: ");  
    scanf("%d", &pf->matricula);  
    printf("Salario: ");  
    scanf("%f", &pf->salario);  
    printf("No. de dependentes: ");  
    scanf("%d", &pf->num_deps);  
}
```

```
int main() {  
    struct FUNCIONARIO f;  
  
    ler(&f);  
    printf("Matricula: %d\n", f.matricula);  
    printf("Salario : %.2f\n", f.salario);  
    printf("No. Deps : %d\n", f.num_deps);  
}
```



➤ Outras características:

4. Para passar uma estrutura **por referência** devemos usar ponteiros.

```
void ler(struct FUNCIONARIO *pf) {  
    printf("Matricula: ");  
    scanf("%d", &pf->matricula);  
    printf("Salario: ");  
    scanf("%f", &pf->salario);  
    printf("No. de dependentes: ");  
    scanf("%d", &pf->num_deps);  
}
```

```
int main() {  
    struct FUNCIONARIO f;  
  
    ler(&f);  
    printf("Matricula: %d\n",    f.matricula);  
    printf("Salario   : %.2f\n", f.salario);  
    printf("No. Deps  : %d\n",    f.num_deps);  
}
```

	f
matricula	243564
salario	5450.0
num_deps	2

- Se quisermos alocar um vetor de 1000000 inteiros precisamos calcular quantos bytes são necessários, pois a função `malloc()` recebe como parâmetro o **número de bytes desejados**.
- Entretanto, como vimos anteriormente, o tamanho em bytes ocupado por determinado tipo **varia de uma arquitetura para outra**.
- Por exemplo, nas antigas máquinas de 16 bits, um inteiro ocupava 2 bytes, enquanto nas máquinas mais modernas de 32 bits um inteiro ocupa 4 bytes.
- Para evitar erros nesse cálculo, podemos usar o operador `sizeof`, que retorna o tamanho em bytes de um determinado tipo de dado:

`sizeof(int)` → retorna o tamanho (em bytes) ocupado pelo tipo `int`

`sizeof(float)` → retorna o tamanho (em bytes) ocupado pelo tipo `float`

```
#include <stdio.h>
```

```
const int TAM = 1000000;
```

```
int main(void) {  
    int *p;
```

Tenta alocar espaço para 1000000 de inteiros

```
    p = malloc(TAM * sizeof(int));
```

```
    if (p == NULL) {  
        puts("Nao ha memoria suficiente para alocar o vetor");  
        return 1;  
    }
```

Preenche a área alocada com números de 1 a 1000000

```
    for (int i = 0; i < TAM; i++)  
        p[i] = i+1;
```

```
    printf("Agora o vetor de %d inteiros esta com dados !\n", TAM);
```

```
    free(p);
```

Libera a memória

```
    puts("Memoria desalocada com sucesso. Vetor não existe mais.");
```

```
    return 0;
```

```
}
```